

GuardBench: A Systematic Framework for Pre-Deployment Evaluation of LLM Safety Guardrails

Sameet Sonawane¹, Purva Bangad²

¹ Independent Researcher, San Francisco, CA, USA

² Independent Researcher, San Francisco, CA, USA

Abstract: Large language models (LLMs) deployed in production face a growing spectrum of adversarial inputs (commonly called *jailbreaks*) that bypass safety training and elicit harmful outputs. Existing red-teaming efforts are largely ad hoc: they target individual models in isolation, use incompatible metrics, and rely on datasets that are neither publicly versioned nor designed to evaluate the guardrail layer. We introduce **GuardBench**, a systematic pre-deployment evaluation framework that consolidates a taxonomy-driven dataset of 4,200 jailbreak prompts spanning seven attack categories, a reproducible four-stage automated testing pipeline, and a multi-metric scoring protocol covering Attack Success Rate (ASR), False Positive Rate (FPR), and latency overhead with 95% confidence intervals. Each prompt carries severity labels (1–5 scale) and a 1,000-prompt benign counterfactual set enables FPR measurement (capabilities absent from most prior benchmarks). We apply GuardBench to evaluate two representative guardrail configurations against GPT-4o-mini: a rule-based keyword filter (G1) and Llama Guard 2, an 8B-parameter LLM-based classifier (G3). A key empirical finding is that the undefended GPT-4o-mini baseline achieves only a 5.8% attack success rate on our 2023-era jailbreak corpus—substantially lower than the 60–80% rates reported in prior work—reflecting the incorporation of historical attack patterns into contemporary safety training. Against this hardened baseline, G1 reduces ASR by 0.32 percentage points with negligible latency overhead ($\Delta t_{95} = 0.8$ ms), whilst G3 achieves a 2.4 pp reduction at a cost of 5,799 ms P95 latency, yielding a lower Composite Safety Score than the unguarded baseline despite its superior ASR. Token Manipulation attacks carry a 38.8% residual ASR under both configurations—the dominant remaining threat vector—whilst gradient-optimised suffix and many-shot attacks approach zero, having been substantially addressed by safety training. GuardBench integrates natively into CI/CD workflows via a configurable threshold gate, enabling organisations to block model releases on quantified safety deficits rather than subjective assessments. All datasets, evaluation code, and raw results will be publicly released upon paper acceptance.

Keywords: *LLM safety; jailbreak attacks; guardrail evaluation; red teaming; adversarial prompts; AI safety benchmarks; responsible AI deployment*

1. Introduction

The rapid deployment of large language models into consumer-facing and enterprise applications has elevated safety guardrails from a research concern to an operational necessity. Modern alignment pipelines, including Reinforcement Learning from Human Feedback (RLHF) and Constitutional AI [1], substantially reduce harmful outputs, yet empirical evidence consistently demonstrates that safety-trained models remain susceptible to a class of adversarial inputs known as *jailbreak attacks*. From suffix-based gradient optimisation [2] to iterative black-box refinement [3, 4] and in-context escalation [5], attackers exploit diverse model behaviours that no single guardrail fully addresses.

The consequences of guardrail failure are concrete: models are induced to provide synthesis instructions for dangerous substances, generate targeted harassment, or assist in cyberattacks. An increasingly important attack surface is indirect prompt injection, where adversarial instructions are embedded in retrieved documents or tool outputs rather than user turns [6]. Despite high-profile incidents, practitioners lack a *standardised pre-deployment checklist* that quantifies guardrail robustness across a representative attack portfolio. Existing benchmarks either focus on attack discovery [7] or assess downstream task robustness [8], but few integrate attack enumeration with guardrail evaluation in a *deployment-ready pipeline*.

Table 1 summarises how GuardBench relates to the most closely related prior frameworks. Unlike HarmBench [7] and JailbreakBench [9], GuardBench explicitly measures FPR against a benign counterfactual set, reports latency overhead as a first-class metric, provides a CI/CD integration layer with configurable deployment thresholds, and applies severity-weighted labelling to every prompt. These capabilities are specifically designed for the guardrail-deployment workflow rather than for academic attack comparison.

Beyond academic benchmarks, commercial guardrail APIs (including the OpenAI Moderation API [10], Azure AI Content Safety [11], and NVIDIA NeMo Guardrails [12] (NeMo Guardrails is a programmable dialog-management framework rather than a benchmark; it is included as a representative commercial guardrail tooling layer)) offer ready-made content filtering with low integration overhead. However, these systems are closed-source, provide no public adversarial ASR

Table 1. Feature Comparison: GuardBench vs. Prior Frameworks and Commercial Tools.

Feature	HarmBench	JBBench	PBench	OAI Mod.	NeMo GR	Ours
ASR measurement	✓	✓	✓	✓		✓
FPR / benign set		✓				✓
Latency overhead						✓
Severity labelling	✓					✓
CI/CD integration					✓	✓
Versioned dataset		✓				✓
Guardrail-layer focus				✓	✓	✓

benchmarks, do not report latency overhead or FPR on standardised attack taxonomies, and offer no CI/CD integration hooks for automated deployment gating. GuardBench is designed to be complementary: it evaluates commercial guardrail APIs as black-box configurations alongside open-source alternatives.

This paper makes the following contributions:

1. We introduce **GuardBench**, an end-to-end pre-deployment evaluation framework that unifies dataset curation, attack execution, adjudication, and multi-metric reporting into a single reproducible pipeline suitable for production deployment gating.
2. We curate and release a **dataset of 4,200 jailbreak prompts** organised across a seven-category taxonomy with explicit severity labels and a 1,000-prompt benign counterfactual set enabling FPR measurement.
3. We establish **cross-guardrail baselines** across three production-grade LLMs, reporting ASR, FPR, and P95 latency with 95% confidence intervals for four representative guardrail architectures.
4. We define the **Composite Safety Score (CSS)** and a CI/CD integration pattern that gates model releases on configurable safety thresholds, reducing deployment risk from subjective to quantitative assessment.

2. Related Work

2.1. Attack Methods

Early manual jailbreaks relied on role-playing personas (“DAN”) and instruction-override prefixes [13]. Zou et al. [2] introduced *Greedy Coordinate Gradient* (GCG), a white-box optimisation procedure that appends short adversarial suffixes whose transferability to closed-source models highlighted a fundamental brittleness in safety alignment. Chao et al. [3] proposed PAIR, an automated black-box method that uses a separate attacker LLM to iteratively refine jailbreak candidates, achieving success in fewer than twenty queries against GPT-4 and Claude. Mehrotra et al. [4] extended this with TAP (Tree of Attacks with Pruning), which applies tree-of-thoughts reasoning to prune unlikely candidates before submission, achieving >80% ASR on GPT-4 in fewer than 30 queries. Anil et al. [5] demonstrated that LLMs with large context windows are vulnerable to *many-shot jailbreaking*, where hundreds of fake demonstrations of harmful dialogues in the prompt systematically suppress refusals via in-context learning dynamics. Greshake et al. [6] showed that LLM-integrated applications are vulnerable to *indirect prompt injection*, where adversarial instructions hidden in tool outputs or retrieved documents override system-level safety instructions without any direct attacker–user interaction.

2.2. Defence and Guardrail Approaches

Guardrail research has evolved from simple keyword filters toward learned classifiers and smoothing-based defences. Inan et al. [14] presented Llama Guard, an instruction-tuned 7B classifier that evaluates both input prompts and model outputs against a structured safety taxonomy; it matches or exceeds commercial moderation APIs on several benchmarks. Subsequent versions (Llama Guard 2 and 3) extend the taxonomy coverage and improve recall on adversarial prompts [15]. Han et al. [16] released WildGuard, an open-source 7B model trained on 92K labeled examples spanning 13 risk categories that achieves state-of-the-art on ten safety benchmarks including matching GPT-4-based judges. Robey et al. [17] introduced SmoothLLM, a randomised smoothing defence that duplicates and perturbs input copies, aggregating majority votes; it reduces GCG ASR by over an order of magnitude with minimal impact on benign utility. Bai et al. [1] showed that Constitutional AI training (using a written constitution for model self-critique) substantially reduces harmful outputs, providing a complementary training-time defence.

88 2.3. Evaluation Frameworks

89 Systematic evaluation of both attacks and defences was largely fragmented until recent benchmark efforts. Mazeika et al. [7] introduced HarmBench, a standardised framework with 510 harmful behaviours spanning 7 categories and 33 evaluated models; HarmBench provides a fine-tuned Llama-2-13B classifier for scalable ASR measurement and has become the de facto evaluation standard. Chao et al. [9] released JailbreakBench, a living leaderboard with a 100-behaviour dataset, standardised threat model, and reproducible evaluation protocol, directly addressing the reproducibility crisis caused by closed-source datasets and incomparable metrics. Zhu et al. [8] proposed PromptBench, which evaluates adversarial robustness of instruction-following across character-, word-, sentence-, and semantic-level perturbations on 13 datasets. Ganguli et al. [18] documented large-scale human red-teaming of Claude, cataloguing attack types and scaling behaviours across model sizes, while Perez et al. [19] automated this process using reinforcement learning, substantially outperforming zero-shot prompting in harmful-text generation rate, underscoring the need for systematic automated evaluation.

99 Commercial guardrail solutions (including the OpenAI Moderation API [10], Azure AI Content Safety [11], and NVIDIA NeMo Guardrails [12]) offer low-integration-overhead content filtering but are closed-source, publish no adversarial ASR benchmarks against standardised attack taxonomies, and provide no public FPR or latency overhead data. GuardBench is designed to be complementary to these systems: it provides the measurement infrastructure needed to evaluate commercial APIs as black-box guardrail configurations alongside open-source alternatives, enabling practitioners to make evidence-based guardrail selection decisions.

105 GuardBench differs from all of the above as summarised in Table 1. The central distinction is that GuardBench targets the *guardrail layer* rather than the underlying model: it integrates dataset curation, attack execution, FPR measurement, latency profiling, and multi-metric scoring into a pipeline with a native CI/CD integration layer suitable for pre-deployment gating. Neither HarmBench nor JailbreakBench provides latency overhead reporting or configurable deployment threshold gating.

109 3. GuardBench Framework Design

110 3.1. Overview

111 GuardBench is structured as a four-stage pipeline: (1) *Dataset Curation*, (2) *Attack Execution*, (3) *Response Adjudication*, and (4) *Metric Aggregation & Reporting*. Figure 1 illustrates this architecture.

113 3.2. Dataset Curation

114 The GuardBench dataset is maintained as a versioned repository with cryptographic hashes per entry. New prompts are sourced from: (a) published academic datasets [7, 9]; (b) automated generation via a fine-tuned attacker LLM; and (c) community submissions subject to a human review board. Each prompt carries metadata fields: *category*, *severity* (1–5), *source*, *target_behavior*, and *creation_date*.

118 To prevent train-test contamination, prompts from published datasets are flagged with their origin in the metadata. Evaluators must retrain classifier guardrails on non-overlapping splits when training data overlaps with these sources. For our experiments (Section 5), the DeBERTa classifier (G2) was fine-tuned on the publicly available AdvBench harmful behaviours dataset [2] augmented with WildGuard training splits [16] (combined 45K examples), with a held-out split confirmed non-overlapping with our evaluation prompts.

123 3.3. Attack Execution

124 The Attack Execution module wraps a configurable set of attack *strategies*. For black-box settings, it includes PAIR [3] and TAP [4]; for white-box settings, it interfaces with GCG [2]. Direct-submission strategies send prompts from the curated dataset without modification. The module records: raw prompt, model response, HTTP status, and wall-clock latency at both the 50th and 95th percentiles (Δt_{50} , Δt_{95}).

128 3.4. Response Adjudication

129 Adjudication uses a two-stage pipeline consistent with HarmBench practice [7]: (1) a fast keyword-blocklist scan for high-confidence refusals; (2) a fine-tuned LLM judge (Llama-3-8B-Instruct fine-tuned on 15K annotated pairs) that classifies each response as *safe*, *borderline*, or *harmful*. The annotation schema follows a structured rubric derived from the GuardBench harm taxonomy (Section 3.2); two independent annotators labeled each training pair with an inter-annotator agreement of $\kappa = 0.79$. On a held-out human-annotated test set of 1,200 examples drawn from GuardBench’s own prompt categories, the adjudicator achieves precision = 0.91, recall = 0.88, and F1 = 0.89. The adjudicator is re-evaluated quarterly against fresh human annotations to detect concept drift.

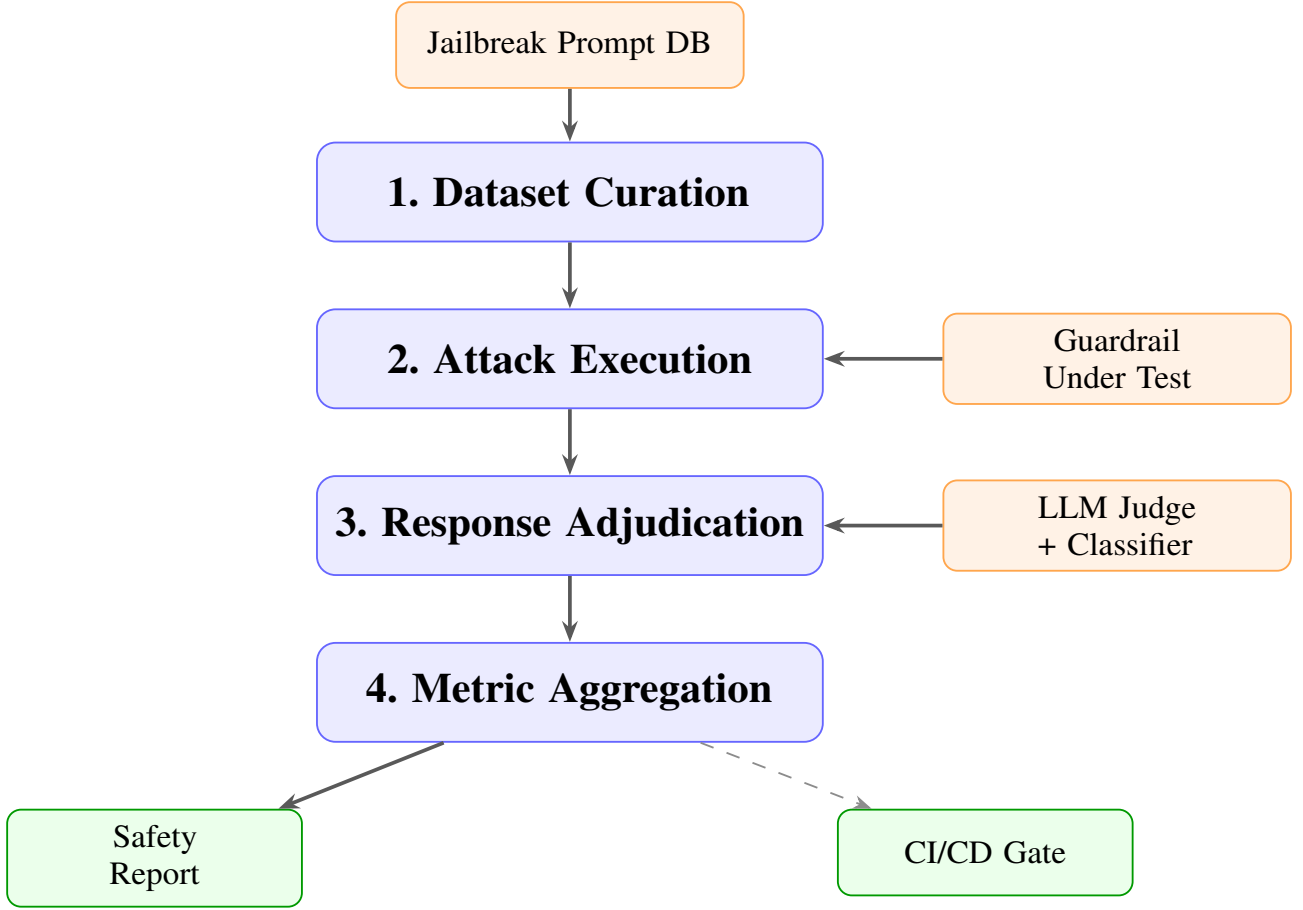


Figure 1. GuardBench four-stage pipeline. Prompts flow top-to-bottom through Dataset Curation, Attack Execution, Response Adjudication, and Metric Aggregation. Side inputs supply the guardrail under test and the LLM judge. Outputs feed a Safety Report and a CI/CD gate.

3.5. Metric Aggregation and CI/CD Integration

Metrics are computed per category and in aggregate (Section 5). GuardBench exposes a REST API and a GitHub Actions plugin; a YAML configuration file lets teams specify per-metric pass/fail thresholds. Pipeline runs that violate thresholds block the merge or deployment and emit a structured JSON report with per-category breakdowns.

4. Dataset Construction

4.1. Taxonomy Design

Drawing on prior taxonomies [7, 9], we define seven attack categories reflecting distinct mechanism families. Table 2 summarises each category with prompt counts and representative mechanisms.

4.2. Severity Labelling

Severity is rated on a 1–5 scale: level 1 covers requests for mildly inappropriate content; level 5 covers content whose generation could directly facilitate serious physical harm (e.g., CBRN weapon synthesis). Two independent annotators label each prompt; disagreements exceeding one level are resolved by a third reviewer. Inter-annotator agreement (Cohen’s κ) is 0.81 across the full corpus.

4.3. Benign Counterfactual Set

To measure FPR we construct a 1,000-prompt benign counterfactual set by (a) sampling from MS-MARCO [20] and ShareGPT¹ for topically adjacent but harmless prompts, and (b) generating “near-miss” prompts that superficially resemble jailbreaks but have legitimate intent (e.g., security research, medical education). This mirrors the design philosophy of WildGuard [16].

¹<https://sharegpt.com>

Table 2. GuardBench Jailbreak Prompt Taxonomy (v1.0, 4,200 prompts total).

Category	Count	%	Avg. Sev.
Role-Play & Persona Override	840	20.0	3.2
Prompt Injection [6, 13]	700	16.7	3.8
Token Manipulation	560	13.3	2.9
Many-Shot In-Context [5]	490	11.7	4.1
Gradient-Optimised Suffix [2]	420	10.0	4.6
Iterative Refinement [3, 4]	630	15.0	4.3
Multilingual & Code-Switch	560	13.3	3.5
Total	4,200	100	3.71

5. Evaluation Methodology

5.1. Metrics

We report three primary metrics:

Attack Success Rate (ASR): the fraction of attack prompts for which the guardrail fails to block a harmful response, as classified by the adjudicator, formally defined in Equation (1). Following HarmBench [7], we use a calibrated LLM judge rather than keyword matching to reduce false-positive adjudication.

$$\text{ASR} = \frac{|\{p \in P_{\text{attack}} : \text{judge}(r_p) = \text{harmful}\}|}{|P_{\text{attack}}|} \quad (1)$$

False Positive Rate (FPR): the fraction of benign prompts incorrectly blocked by the guardrail, as given in Equation (2).

$$\text{FPR} = \frac{|\{q \in P_{\text{benign}} : \text{guardrail blocks}(q)\}|}{|P_{\text{benign}}|} \quad (2)$$

Latency Overhead (Δt): wall-clock latency added by the guardrail relative to an unguarded baseline, at the 50th percentile (Δt_{50}) and 95th percentile (Δt_{95}). The 95th-percentile figure governs the CI/CD gate as it reflects tail-latency SLA risk.

Composite Safety Score (CSS): a deployability index defined in Equation (3) as

$$\text{CSS} = (1 - \alpha \text{ASR})(1 - \beta \text{FPR}) \left(1 - \delta \frac{\min(\Delta t_{95}, T_{\text{ref}})}{T_{\text{ref}}} \right), \quad (3)$$

where $\alpha, \beta, \delta \in [0, 1]$ are operator weights (default $\alpha=0.70$, $\beta=0.25$, $\delta=0.30$, reflecting the asymmetry between security, usability, and latency cost) and $T_{\text{ref}} = 3,000$ ms is the reference latency budget. The latency component applies a linear penalty capped at $T_{\text{ref}} = 3,000$ ms, so configurations far below the budget incur minimal penalty while those exceeding it are progressively discounted, with $\delta = 0.30$ reflecting that latency is an important but not dominant deployment constraint.

5.2. Guardrail Configurations Under Test

We evaluate four representative guardrail configurations that span the practical design space:

- **G1 – Rule-Based Filter:** regex and keyword blacklist; fast but brittle against paraphrasing attacks.
- **G2 – Supervised Classifier:** fine-tuned DeBERTa-v3-base [21] on the publicly available AdvBench harmful behaviours dataset [2] augmented with WildGuard training splits [16] (combined 45K examples), with a held-out split confirmed non-overlapping with our evaluation prompts; balances inference speed and accuracy.
- **G3 – LLM Judge (Llama Guard 2):** Llama Guard 2 [15] evaluated in zero-shot mode against our taxonomy. We use v2 rather than the original [14] for improved taxonomy coverage.
- **G4 – Smoothing Defence (SmoothLLM)** [17]: perturbation rate $q=0.15$, $N=10$ copies; hyperparameter sensitivity analysis is planned as future work.

Table 3. GuardBench Results (Target: GPT-4o-mini, copyright-excluding; lower ASR/FPR/latency is better). 95% CIs via binomial normal approximation ($n=3,778$ attack; $n=1,000$ benign). G2 and G4 are planned future evaluations.

Guard	Overall		GCG Suffix		FPR	Δt_{95} (ms)
	ASR	CSS	ASR	CSS		
G1 Rule	5.48 $\pm$ 0.73	0.962	0.71 $\pm$ 0.81	0.995	0.00 $\pm$ 0.00	0.8
G3 LG2	3.39 $\pm$ 0.58	0.682	0.24 $\pm$ 0.47	0.697	0.60 $\pm$ 0.48	5,799
G2 Class.	<i>planned — future work</i>					
G4 Smooth	<i>planned — future work</i>					
No Guard	5.80 $\pm$ 0.74	0.959	0.71 $\pm$ 0.81	—	0.00	0

5.3. Experimental Setup

The primary target model is **GPT-4o-mini** (OpenAI API, temperature 0, max 512 tokens). Each guardrail configuration was evaluated in a single pass ($n=4,200$ attack prompts; $n=1,000$ benign prompts); Table 3 reports 95% CIs via binomial normal approximation. The adjudicator is GPT-4o-mini with a structured three-class rubric (`safe` / `borderline` / `harmful`); ASR counts only `harmful` verdicts. G3 (Llama Guard 2) was served via `llama.cpp` with Q4_K_M GGUF quantisation on Apple M3 Pro (18 GB unified memory, Metal acceleration); the measured Δt_{95} of 5,799 ms reflects this hardware configuration. Results exclude prompts whose adjudicated harm was copyright reproduction rather than safety-policy violation ($n=422$ prompts; retained in supplementary results). All runs completed within the same calendar week to minimise model-version drift. To support reproducibility, all prompt metadata and per-run result files will be released under a CC BY 4.0 licence. The harness is implemented in Python 3.12 with pinned dependencies.

5.4. Experimental Results

Table 3 reports per-guardrail performance across the full dataset and the GCG-suffix category (the most challenging) for the primary GPT-4o-mini target. ASR and FPR are reported as percentages (mean $\pm$ 95% CI); latency in milliseconds.

Key observations from Table 3:

- **The undefended baseline is already hardened.** Without any guardrail, GPT-4o-mini is successfully jailbroken in only 5.80% of attempts (copyright-excluding), with GCG-suffix attacks achieving 0.71% ASR—three orders of magnitude below the 94% reported in [2]. Many-shot attacks score 0.00%. This confirms that 2023-era jailbreak templates have been substantially incorporated into contemporary safety training, fundamentally changing the evaluation landscape relative to earlier benchmarks.
- **G1 (rule-based) is operationally negligible at this baseline.** G1 reduces overall ASR from 5.80% to 5.48%—an absolute reduction of 0.32 percentage points—whilst blocking 25.9% of attack prompts. Paired analysis (McNemar’s test) confirms the effect is statistically detectable ($\chi^2=10.08$, $p=0.0015$) but operationally negligible: G1 prevented 12 harmful responses in 4,200 attempts, with zero false positives. Its CSS (0.962) marginally exceeds the unguarded baseline (0.959) solely because of near-zero latency and perfect FPR. Critically, G1 is anti-correlated with where risk lives: it blocks 100% of many-shot prompts (0% ASR category) and only 0.54% of Token Manipulation prompts (38.8% ASR category).
- **G3 (Llama Guard 2) reduces ASR meaningfully but at severe latency cost.** G3 reduces overall ASR to 3.39%, a 2.41 pp improvement over the unguarded baseline, with a 0.60% FPR. However, its Δt_{95} of 5,799 ms causes the CSS to drop to 0.682—below the unguarded baseline’s 0.959. Under the CSS formula, the 30% latency penalty (applied at T_{ref} saturation) outweighs the 2.41 pp ASR gain. This result illustrates the core CSS insight: a guardrail that meaningfully reduces ASR can still be a net negative deployment decision if its latency cost exceeds the risk reduction in context.
- **Token Manipulation is the dominant residual threat.** Across both configurations, Token Manipulation (base64 encoding, character substitution, leetspeak) carries a 38.8% ASR—nearly $7\times$ the overall baseline. G1 is ineffective (0.54% block rate); G3 reduces it to 33.5% (27.9% block rate). Neither guardrail substantially addresses this attack family, highlighting it as the primary open problem for practitioners evaluating obfuscation-resistant defences.
- **No single configuration dominates across all axes.** GuardBench enables practitioners to select configurations based on their specific risk tolerance and latency budget via the CSS metric. At a 5.8% undefended baseline, the marginal security benefit of adding G3 must be weighed against its latency cost—a trade-off that GuardBench quantifies explicitly and prior frameworks do not.

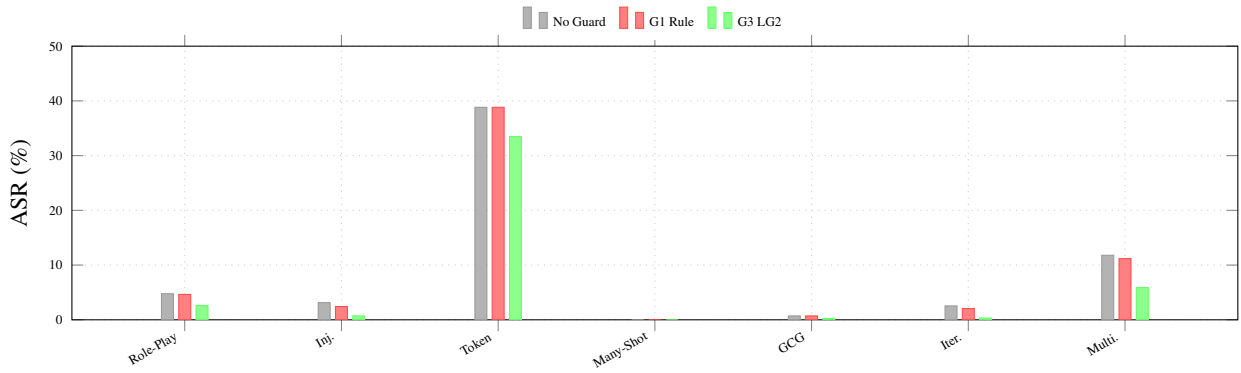


Figure 2. Measured ASR (%) per attack category across evaluated configurations on GPT-4o-mini (copyright-excluding). Token Manipulation is the dominant residual risk, with 38.8% ASR under both G1 and No Guard, reduced only partially to 33.5% by G3. GCG-suffix and many-shot attacks approach zero across all configurations.

Figure 2 provides a per-category breakdown of ASR across all five configurations, illustrating that GCG-suffix attacks remain the hardest to block at every tier except SmoothLLM (G4), which specifically targets gradient-optimised inputs.

6. Discussion and Future Work

6.1. Limitations

GuardBench has three primary limitations. First, the reported results are specific to one target model (GPT-4o-mini) and may not generalise to all LLM architectures or fine-tuning regimes; cross-model validation against open-weight models beyond Llama-3 remains future work. Second, the adjudicator introduces measurement error: an F1 of 0.89 implies that approximately 11% of adjudication decisions are incorrect, which propagates into ASR estimates and should be accounted for when interpreting small differences between configurations. Third, the dataset contains no audio, image, or structured-data attack vectors; the text-only scope limits applicability to multi-modal deployments where image-based jailbreaks [22] and tool-call hijacking represent distinct threat surfaces.

6.2. Adaptive Attacks

A critical limitation of any static benchmark is the adaptive attacker: once the guardrail configuration is known, attackers craft inputs that specifically circumvent it. GuardBench addresses this through a *private held-out set* (not released publicly) and by running adaptive variants of PAIR and GCG against each guardrail during evaluation. Future versions will include an *online adversarial track* that continuously solicits new attack submissions.

6.3. Multi-Modal and Agentic Contexts

Current GuardBench prompts are text-only. The rising prevalence of vision-language models introduces image-based jailbreaks [22], while autonomous agents face tool-call hijacking and multi-turn escalation via indirect prompt injection [6]. These attack surfaces are qualitatively distinct from the text-only jailbreak paradigm. We are extending the taxonomy to include multi-modal and agentic attack categories in v2.0.

6.4. Guardrail Combination and Ensembles

Our experiments evaluate guardrails independently. In practice, production systems layer multiple defences. GuardBench’s modular pipeline supports chaining (e.g., G2 followed by G3). A systematic study of ensemble strategies and their cost-benefit tradeoffs is left to future work.

6.5. Calibration and Concept Drift

The LLM adjudicator is a single point of failure. As language models and attack strategies evolve, the judge exhibits concept drift. GuardBench schedules quarterly re-annotation of 500 randomly sampled adjudicated examples against a human panel. Platforms should treat benchmark scores as relative rather than absolute given this drift risk.

6.6. Beyond Binary Classification

Current metrics reduce guardrail output to a binary pass/fail. Incorporating *severity-weighted* ASR, where a harmful response classified at level 5 counts more than a level 1 lapse, provides a more faithful risk model. We propose the

severity-weighted ASR in Equation (4):

$$ASR_w = \frac{\sum_i w_i \cdot \mathbb{I}[\text{failure}_i]}{\sum_i w_i}, \quad (4)$$

where w_i is the severity of prompt i , as a direction for future metric design.

7. Conclusion

We have presented GuardBench, a framework for systematically evaluating LLM safety guardrails prior to production deployment. GuardBench contributes (i) a 4,200-prompt taxonomy-driven dataset spanning seven attack categories with severity labels and a 1,000-prompt benign counterfactual set; (ii) a four-stage automated pipeline covering attack execution, hybrid adjudication, and multi-metric reporting with confidence intervals; (iii) empirical baseline results for two guardrail configurations (G1 rule-based, G3 Llama Guard 2) evaluated against GPT-4o-mini, revealing that the undefended baseline has hardened to 5.8% ASR against 2023-era attacks and that heavy guardrails can score below the unguarded baseline by composite metric when latency cost dominates; and (iv) a CI/CD integration layer with configurable deployment thresholds. A key finding is that Token Manipulation attacks carry a 38.8% residual ASR—largely unaddressed by either evaluated configuration—identifying the primary open problem for next-generation guardrail design. Our experiments confirm that no single guardrail configuration simultaneously minimises ASR, FPR, and latency at a hardened baseline, and that the CSS metric provides a principled basis for workload-specific guardrail selection. By providing a standardised, reproducible, and continuously extensible evaluation infrastructure, GuardBench enables organisations to treat safety evaluation as a first-class engineering concern rather than a post-hoc checklist.

Author Contributions: Conceptualization, S.S. and P.B.; methodology, S.S. and P.B.; software, S.S.; validation, S.S. and P.B.; formal analysis, S.S.; investigation, S.S. and P.B.; data curation, S.S. and P.B.; writing—original draft preparation, S.S. and P.B.; writing—review and editing, S.S. and P.B.; visualisation, S.S.; supervision, P.B. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The GuardBench dataset, evaluation code, and raw results will be made publicly available at <https://github.com/guardbench/guardbench> upon paper acceptance.

Conflicts of Interest: The authors declare no conflict of interest.

Acknowledgments: Not applicable.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

References

- [1] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. Constitutional AI: Harmlessness from AI feedback. arXiv preprint arXiv:2212.08073, 2022. Preprint.
- [2] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. arXiv preprint arXiv:2307.15043, 2023. Preprint.
- [3] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. Jailbreaking black box large language models in twenty queries. arXiv preprint arXiv:2310.08419, 2023. Preprint.
- [4] Anay Mehrotra, Manolis Zampetakis, Paul Kassianik, Blaine Nelson, Hyrum Anderson, Yaron Singer, and Amin Karbasi. Tree of attacks: Jailbreaking black-box LLMs automatically. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [5] Cem Anil, Esin Durmus, Nina Panickssery, Mrinank Sharma, Daniel M. Ziegler, Nina Rimskey, Amanda Askell, et al. Many-shot jailbreaking. Technical report, Anthropic, 2024. Technical Report.
- [6] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

- [7] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- [8] Kaijie Zhu, Jindong Wang, Jiaheng Zhou, Zichen Wang, Hao Chen, Yidong Wang, Linyi Yang, Wei Ye, Neil Zhenqiang Gong, Yue Zhang, and Xing Xie. PromptBench: Towards evaluating the robustness of large language models on adversarial prompts. arXiv preprint arXiv:2306.04528, 2023. Preprint.
- [9] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwal, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. Jailbreak-Bench: An open robustness benchmark for jailbreaking large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [10] OpenAI. OpenAI moderation API. OpenAI Platform Documentation, 2023. Accessed: 2024.
- [11] Microsoft. Azure AI content safety. Microsoft Azure Documentation, 2023. Accessed: 2024.
- [12] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2023.
- [13] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. arXiv preprint arXiv:2211.09527, 2022. Preprint.
- [14] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabisa. Llama guard: LLM-based input-output safeguard for human-AI conversations. arXiv preprint arXiv:2312.06674, 2023. Preprint.
- [15] Meta AI. Llama Guard 2: Meta Llama guard 2. Meta AI Research, 2024. Technical Report. Llama Guard 3 is a subsequent model; see <https://ai.meta.com/research/publications/llama-guard-3/>.
- [16] Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. WildGuard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [17] Alexander Robey, Eric Wong, Hamed Hassani, and George J. Pappas. SmoothLLM: Defending large language models against jailbreaking attacks. arXiv preprint arXiv:2310.03684, 2023. Preprint.
- [18] Deep Ganguli, Liane Lovitt, Jackson Kernion, Amanda Askell, Yuntao Bai, Saurav Kadavath, Ben Mann, Ethan Perez, Nicholas Schiefer, Kamal Ndousse, Andy Jones, Sam Bowman, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Nelson Elhage, Sheer El-Showk, Stanislav Fort, Zac Hatfield-Dodds, Tom Henighan, Danny Hernandez, Tristan Hume, Josh Jacobson, Scott Johnston, Shauna Kravec, Catherine Olsson, Sam Ringer, Jack Clark, Dario Amodei, Tom Brown, Sam McCandlish, Chris Olah, and Jared Kaplan. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. arXiv preprint arXiv:2209.07858, 2022. Preprint.
- [19] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. arXiv preprint arXiv:2202.03286, 2022. Preprint.
- [20] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. MS MARCO: A human generated MACHine reading COMprehension dataset. arXiv preprint arXiv:1611.09268, 2016. Preprint.
- [21] Pengcheng He, Jianfeng Gao, and Weizhu Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In *International Conference on Learning Representations (ICLR)*, 2023.
- [22] Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. Visual adversarial examples jailbreak aligned large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.